

0.52. Deux carrés à la suite

Deux appels à `carre`, l'un après l'autre : un petit carré lent, puis un grand carré rapide. Le second attend la fin du premier.

1. Ce qu'il faut comprendre

Deux carrés différents, l'un après l'autre. Chacun a été vu seul dans les leçons d'avant ; on les enchaîne.

Ce qui est nouveau ici : deux `carre` l'un après l'autre. `carre` **bloque** : elle attend la fin de son carré avant de rendre la main. La deuxième ligne ne commence donc que quand la lettre est revenue au centre.

Le premier est celui du 0.34 (taille 1). **Le second** change quatre réglages, chacun vu dans sa leçon : taille 3 (le 0.36), sens -1 (le 0.42), vitesse 100 (le 0.44), 3 tours (le 0.47).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <carre>` → [0.33.3. La fonction `carre\(\)` — tourner en carré](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1380 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
carre()         440 o ← main() – une fonction fournie par le langage
main()          69 o ← démarrage – ta fonction
VBlank          8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage   28 o ← image() – attend l'image suivante
AttendreVBlank  16 o ← démarrage – attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() – efface des cases
CopierTuiles    18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage – vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()  × 4   52 o sur place, dans carre() + AdresseCase (10 o)
effacer() × 3   45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image()  × 4   12 o sur place, dans carre(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
carre()  × 2   62 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <carre>      // une tuile qui tourne en carré
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

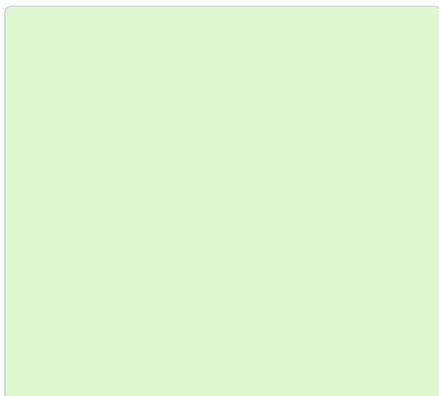
```
int main() {
    // 1. Le petit carré (le 0.34) : taille 1, sens 1, 250 ms, 1 tour.
    carre(10, 8, ALPHABET[0], 1, 1, 250, 1);

    // 2. Le grand carré. Il commence quand le petit est FINI :
    //    carre attend la fin de son tour avant de passer à la ligne suivante.
    //    taille 3, sens -1, vitesse 100, 3 tours
    carre(10, 8, ALPHABET[0], 3, -1, 100, 3);

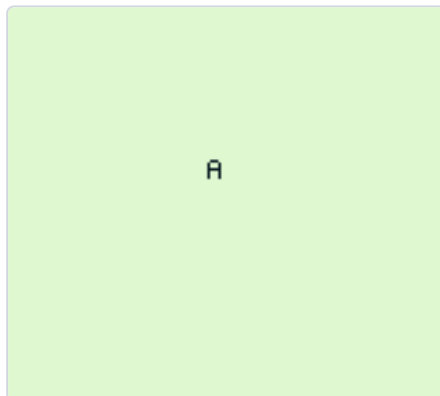
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

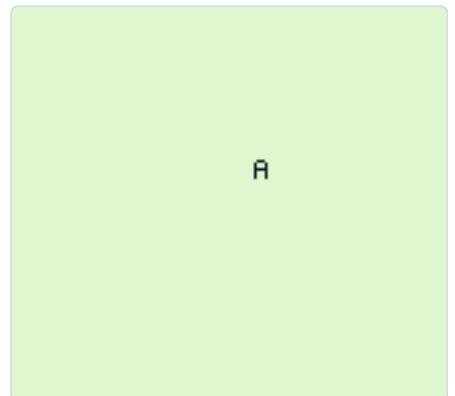
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



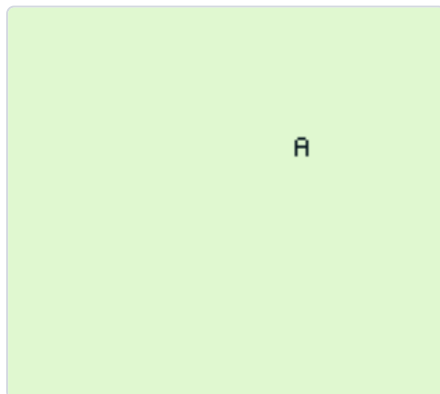
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui fait un petit tour lent autour de (10, 8), puis trois grands tours rapides dans l'autre sens.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le petit carré d'abord : son coin (11, 9)

✓ puis trois grands tours : (13, 11) trois fois

✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.52.1. Deux carrés à la suite — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++